High Level Design Document

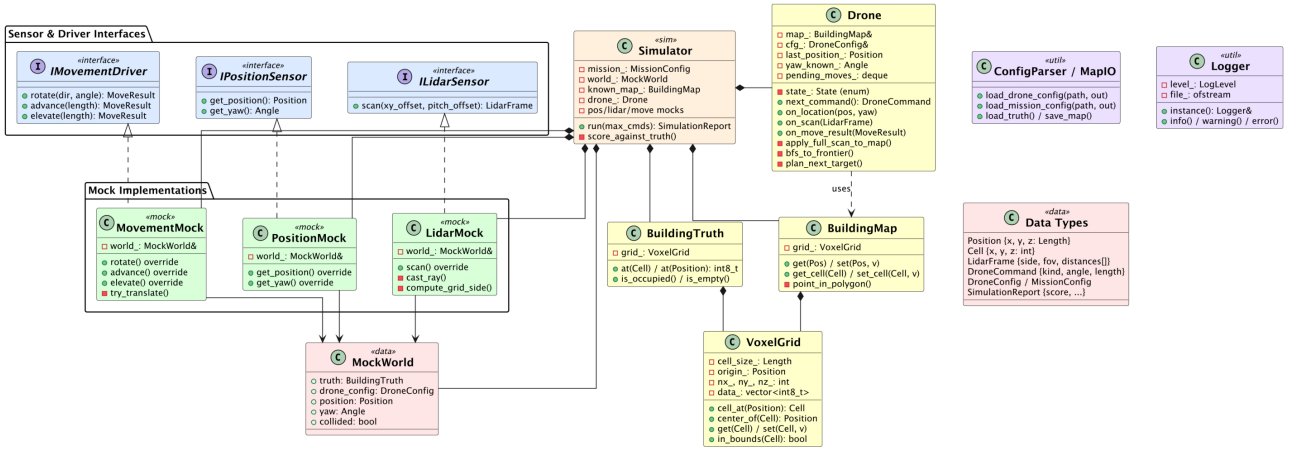
Drone Mapper – Exercise 1

Advanced Topics in Programming, Semester B 2026

Name	ID
Almog Tavor	(ID: 323084962)
Yonatan Kahana	(ID: 212223036)

1. UML Class Diagram

The diagram below shows the main classes, their relationships, and key methods. Interfaces are blue, core classes yellow, mock implementations green, data types pink, and utilities purple.

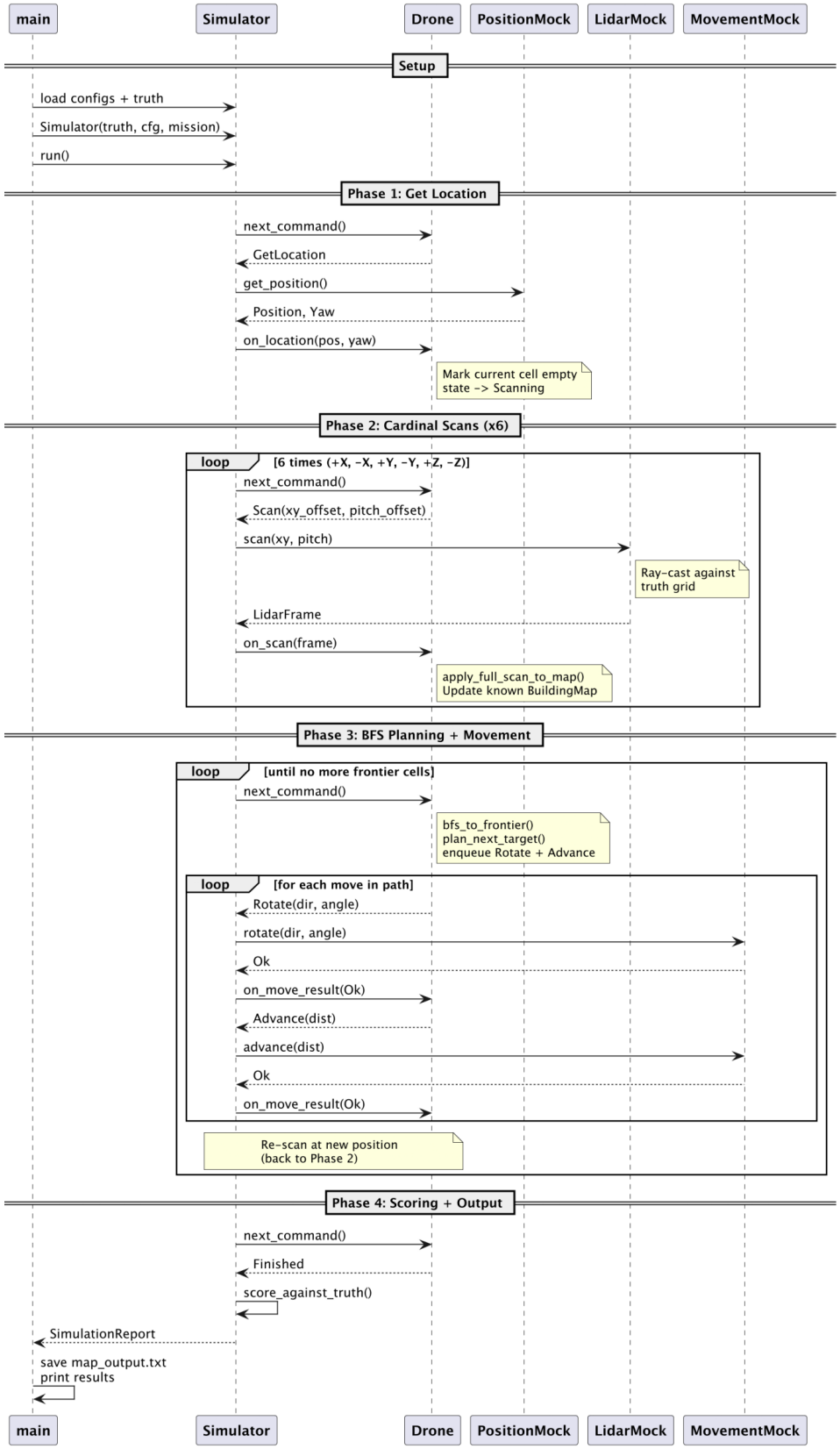


Key relationships:

- **Simulator** owns all mocks, the Drone, both maps, and orchestrates the loop.
- **Drone** sees only abstract interfaces (ILidarSensor, IPositionSensor, IMovementDriver) through the command/response pattern – it never touches the mocks or truth directly.
- **MockWorld** is shared state between the three mocks so that movement updates are visible to the position and lidar sensors.
- **VoxelGrid** is the shared spatial data structure, composed into both BuildingTruth and BuildingMap.

2. UML Sequence Diagram – Main Flow

The diagram shows the four phases of the main simulation loop: location query, scanning, movement, and scoring.



3. Design Considerations and Alternatives

3.1 Separation of Drone Logic from Simulation

The drone never directly accesses the building truth or the mock implementations. Instead, the **Simulator** acts as a mediator: it asks the Drone for a command, dispatches it to the appropriate mock, and feeds the result back. This mirrors a real embedded system where the flight controller issues commands and receives sensor data through hardware abstraction layers.

Alternative considered: Letting the drone call sensor/driver interfaces directly (dependency injection). We rejected this because the assignment's "main loop" specification explicitly requires the simulator to ask the drone for commands, making the command-response pattern the natural fit.

3.2 Voxel Grid as Shared Spatial Representation

Both `BuildingTruth` and `BuildingMap` wrap the same `VoxelGrid` class. This ensures identical coordinate-to-cell mapping and makes the scoring comparison straightforward (iterate both grids in lock-step).

Alternative considered: Using an octree or sparse hash map for the drone's known map (since most cells start as "unmapped"). We chose a dense grid for simplicity and because the building sizes in exercise 1 are small enough that memory is not a concern.

3.3 Full Lidar Frame Back-Projection

Initially we used only the central ray of each cardinal scan to update the map. This left corner cells unmapped and produced scores around 96%. We improved to back-projecting *every* ray in the lidar frame to world coordinates, walking each ray through the voxel grid and marking cells accordingly. This raised scores to 99%+ and reduced command counts by 60%.

Trade-off: The full frame approach is $O(n^2)$ per scan (where n is the lidar grid side), but with our capped grid size ($\leq 51 \times 51$) and only 6 scans per waypoint, the overhead is negligible.

3.4 Frontier-Based BFS Exploration

The drone maintains a "frontier" – known-empty cells adjacent to unmapped cells. At each waypoint it runs BFS through known-empty cells to find the nearest frontier, plans the shortest path, and follows it. This is complete (it will visit every reachable cell) and deterministic (BFS neighbor order is fixed).

Alternative considered: DFS / wall-following. DFS is simpler but can lead to long back-tracking paths. A* with distance heuristic could reduce path lengths but adds complexity without material benefit at exercise 1 grid sizes.

3.5 Strong Types with mp-units

All values and APIs use the mp-units library (v2.0.0) as required by the assignment. A thin wrapper header (`include/units/Units.h`) includes mp-units and provides `Length` and `Angle` type aliases plus `using` declarations for `cm`, `m`, and `deg`. This keeps downstream code concise (`5.0 * cm`, `90.0 * deg`) while mp-units enforces dimensional safety at compile time.

The mp-units and gsl-lite header files are bundled under `third_party/` so that the submission is self-contained and compiles without requiring these libraries to be pre-installed on the build machine.

3.6 Polygon Boundary Support

The mission boundary is defined as an arbitrary polygon (supporting concave shapes like L-buildings). We use the standard ray-casting point-in-polygon algorithm to classify each voxel during `BuildingMap` initialization.

Alternative considered: Axis-aligned bounding box (simpler, but cannot represent non-rectangular buildings). We chose polygon support since the assignment explicitly mentions "polygon of {X, Y}".

3.7 Floating-Point Position Snapping

We discovered that cumulative floating-point drift from `cos(π/2)` not being exactly zero caused the drone's actual position to cross cell boundaries, leading to incorrect lidar readings and infinite loops. The fix: snap world position to the nearest 0.01 cm after every movement in the mock.

3.8 Error Recovery Strategy

All input file errors are recoverable where possible: missing config keys use defaults, malformed map rows are padded, and unknown keys are logged. Only truly fatal errors (file not found, no grid data) cause the program to print an error and exit via `main` return – never via `exit()` or exceptions escaping `main`.

4. Testing Approach

4.1 Framework

We use a custom dependency-free micro test framework (`tests/test_framework.h`) that provides `TEST()`, `CHECK()`, `CHECK_EQ()`, and `CHECK_NEAR()` macros. Tests are auto-registered and run sequentially from a single `main()`.

4.2 Test Layers

Layer	File (count)	What is tested
Unit types	test_units.cpp (4)	Length/Angle construction, arithmetic, normalization, radian conversion
Spatial data	test_voxel_grid.cpp (4)	Set/get, out-of-bounds, continuous-to-cell conversion, origin offset
Known map	test_building_map.cpp (2)	Polygon boundary marking, height-range filtering
Config I/O	test_config_parser.cpp (4)	Full parse, unknown key recovery, polygon vertices, missing file
Map I/O	test_map_io.cpp (3)	Load truth, round-trip save/load, missing file
Lidar mock	test_lidar_mock.cpp (2)	Central ray hit in corridor, open-direction returns -1
Movement mock	test_movement_mock.cpp (4)	Advance direction, rotate yaw, wall collision, max clamping
Drone logic	test_drone.cpp (2)	Initial GetLocation command, scan sequence after location
Integration	test_simulator.cpp (2)	Full simulation of small room ($\geq$80% score); sealed pocket (no collision)
	Total: 27 tests	

4.3 Testing Strategy

- **Bottom-up:** Each module is tested in isolation before integration. Mocks are tested against known truth grids with predictable geometry.
- **Integration:** The two simulator tests create a full truth→simulator→drone pipeline and verify end-to-end behaviour (score threshold, no collisions).
- **Error paths:** Config parser tests verify that missing files, unknown keys, and malformed values are handled gracefully with recoverable defaults.

- **Determinism:** All tests are deterministic (no randomness in the algorithm). Running the same input always produces the same output.
- **Run:** `make test` builds and executes the entire suite.

4.4 Sample Input Sets

Three input sets exercise different scenarios:

1. **set1** – Simple 10×10 empty room (baseline correctness).
2. **set2** – 15×12 building with a sealed 3×3 pocket (tests that inaccessible areas remain unmapped without crashing).
3. **set3** – L-shaped building with non-convex polygon boundary (tests polygon clipping and multi-arm exploration).